INF20325

ESTRUCTURAS DE DATOS

Ayudantía 9: Árboles Rojo-Negro

Ignacia Nettle
inettle@usm.cl

BLOQUE 1

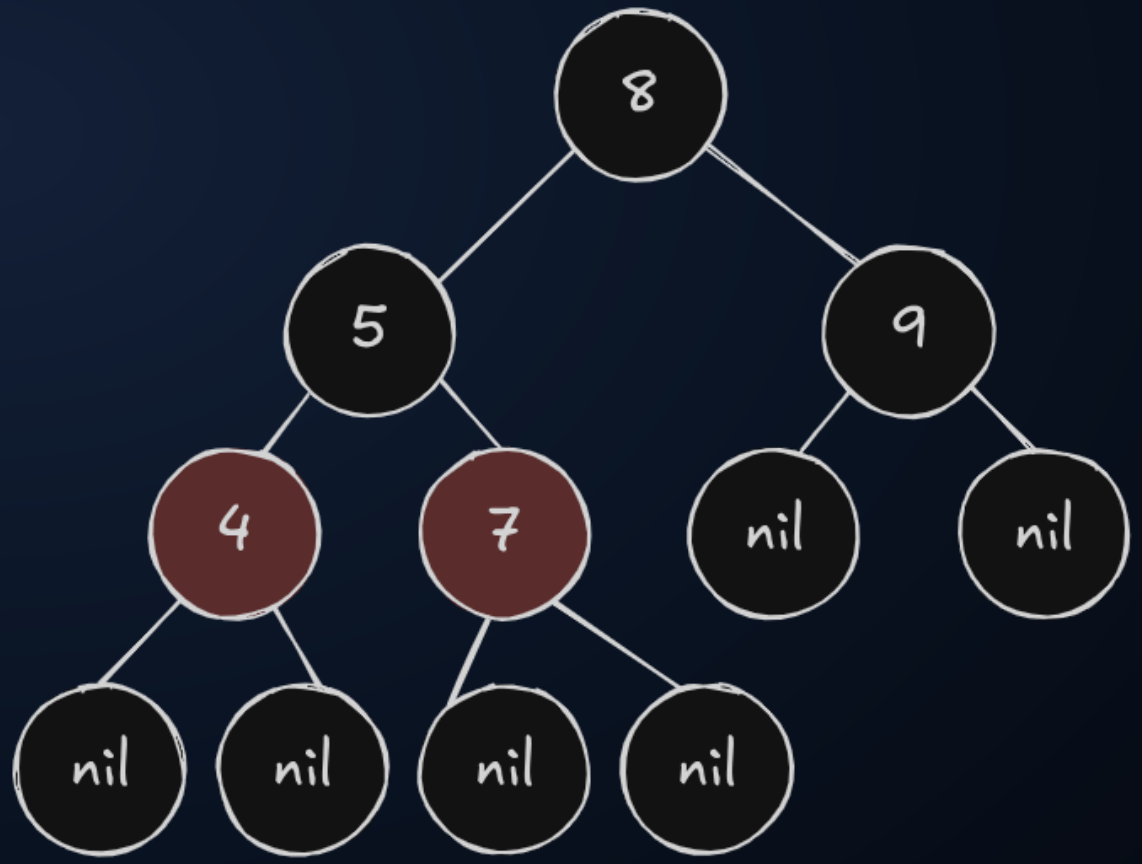
Árboles Rojo-Negro

Árboles Rojo-Negro

Definición

Un árbol rojo-negro es un **árbol binario** ordenado con altura máxima $O(\log n)$, en el cual cada nodo posee un atributo adicional: un **color**, que puede ser rojo o negro.

Los colores tienen como propóstio asegurar una manipulación eficiente de información durante inserción y borrado.



Árboles Rojo-Negro

Propiedades Importantes

- Cada nodo es rojo o negro.
- La raíz **siempre** es negra.
- Los nodos rojos **no pueden tener hijos rojos**.
- Hojas nulas (nil) se consideran negras.
- Cualquier camino desde un nodo hasta sus hojas nulas debe tener la **misma cantidad de nodos negros**.

Otras Propiedades

- Sea h la altura de un árbol rojo-negro con N nodos: $h < 2\log(N+1)$.
- La cantidad de nodos negros de cualquier camino que va desde la raíz a una hoja debe ser $\geq h$.

Árboles Rojo-Negro: `Operaciones Básicas`

Búsqueda

El algoritmo de búsqueda en árboles rojo-negro siguen la misma lógica de un árbol binario:

- Si el elemento buscado es **menor** que el valor del nodo actual, se continúa por su hijo **izquierdo**.
- Si el elemento buscado es **mayor** que el valor del nodo actual, se continúa por su hijo **derecho**.

El valor de retorno de una función de búsqueda debe ser un puntero al nodo encontrado en caso de éxito, o `NULL` en caso de que no se haya encontrado el elemento.

Inserción

La inserción de un nodo nuevo se realiza en dos pasos:

- `Inserción:` el nuevo nodo **siempre es rojo** y se inserta como una hoja.
  - Si el árbol está vacío, se cambia el color del nodo insertado a negro.
- `Reequilibrado:` Si el padre del nuevo nodo también es **rojo**, se deben revisar *iterativamente* tres casos de modo que no quede una configuración **inválida**.

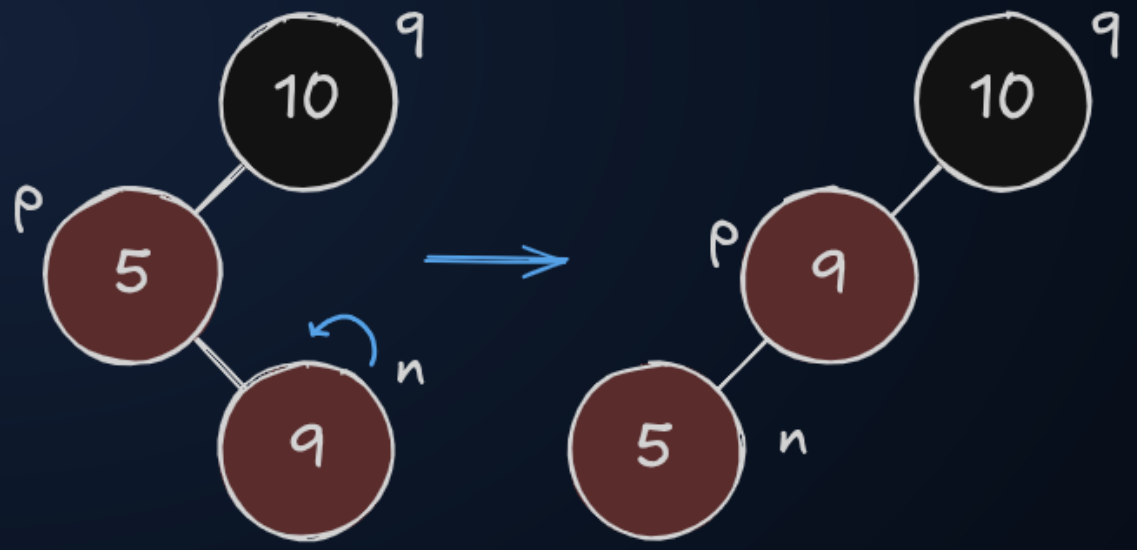
Reequilibrado: `Caso 1`

Se ha inserado un nodo `n`, cuyo padre es `p` y el padre de su padre es `q`, se cumple que:

- `p` y `n` son nodos rojos.
- `p` es hijo izquierdo de `q`.
- `n` es hijo derecho de `p`.
- `q` tiene hijo derecho **negro** o **nulo**.

En este caso, se debe realizar una **rotación izquierda** *(similar a la rotación izqueirda simple en AVL)*.

El procedimiento es **equivalente** para `p` hijo derecho y `n` hijo izquierdo.



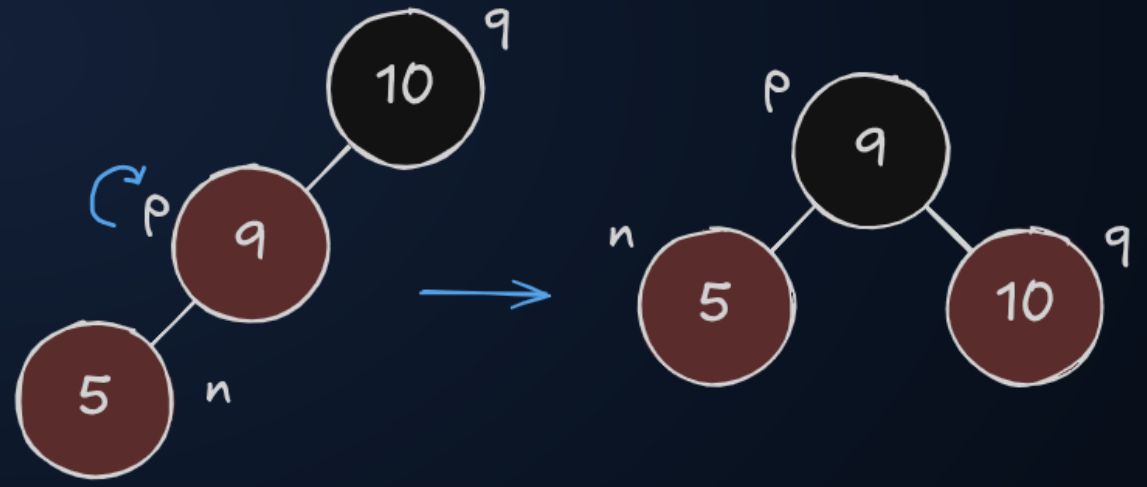
Reequilibrado: Caso 2

Se ha insertdo un nodo `n`, cuyo padre es `p` y el padre de su padre es `q`, se cumple que:

- `p` y `n` son nodos rojos.
- `p` es un hijo izquierdo de `q`.
- `n` es un hijo izquierdo de `p`.
- `q` tiene hijo derecho **nulo** (no tiene).

En este caso, se realiza una **rotación derecha** *(similar a la rotación derecha simple en AVL)*.

El procedimiento es **equivalente** para `p` hijo derecho y `n` hijo derecho.



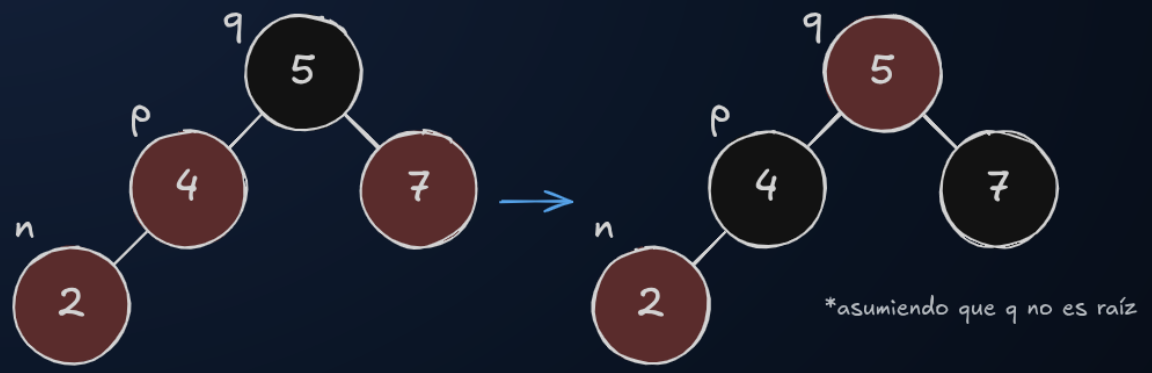
Reequilibrado: Caso 3

Se ha insertado el nodo `n`, cuyo padre es `p` y el padre de su padre es `q`, se cumple que:

- `p` y `n` son nodos rojos.
- `n` es nodo izquierdo o drecho de `p`.
- `p` es nodo izquierdo de `q`.
- `q` tiene un nodo derecho rojo.

En este caso, **no se realizan rotaciones**, sino que basta con cambiar el color de los hijos de `q` a **negro** y el color de `q` a **rojo**. En caso de que `q` sea la raíz, este **se mantiene de color negro**.

El procedimiento es **equivalente** para `p` hijo derecho.

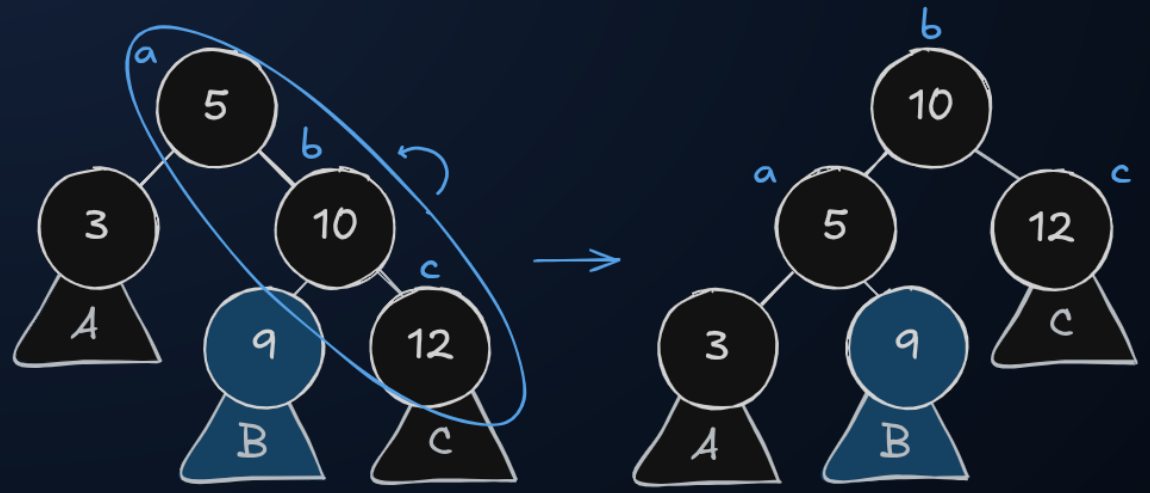


Reequilibrado: `Rotaciones de nodos con sub-árboles`

Rotación Simple Izquierda

Dados los nodos `a`, `b` y `c` que forman parte de una rotación simple izquierda:

- El sub-árbol **izquierdo** de `b` pasa a ser sub-árbol de `a`.

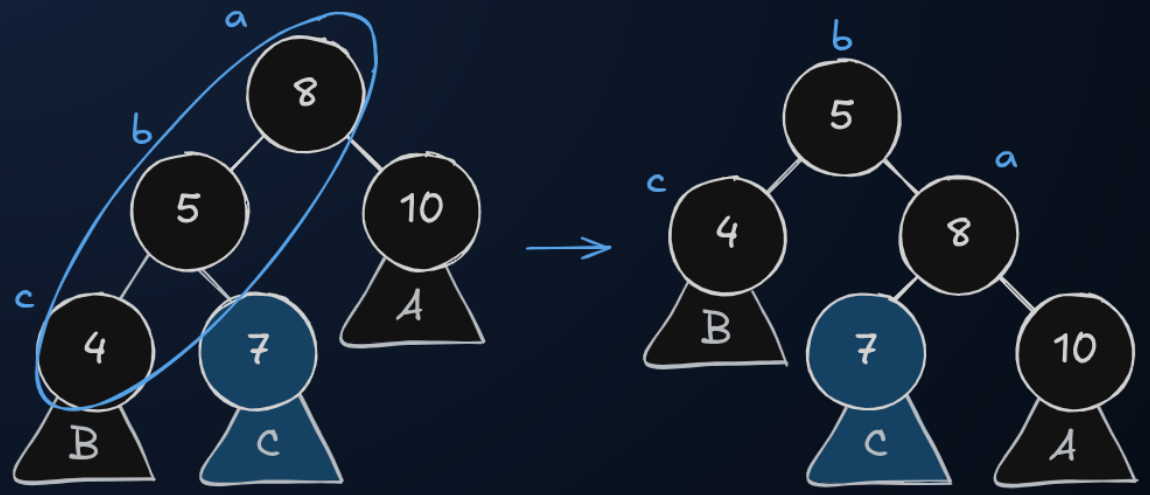


Reequilibrado: Rotaciones de nodos con sub-árboles

Rotación Simple Derecha

Dados los nodos `a`, `b` y `c` que forman parte de una rotación simple derecha:

- El sub-árbol **derecho** de `b` pasa a ser sub-árbol de `a`.



Hint: Estas técnicas también funcionan para las rotaciones simples en AVL

Árboles Rojo-Negro: `Operaciones Básicas`

Borrado

El borrado de un nodo también se realiza en dos pasos:

- `Eliminación:`
  - Si el nodo a eliminar tiene dos hojas nulas, simplemente se remueve.
  - De lo contrario, se intercambia su valor por el del nodo más a la derecha del sub-árbol izquierdo o del más a la izquierda del sub-árbol derecho. Luego, se elimina el nodo hoja correspondiente.
- `Reequilibrado:` Eliminar un nodo rojo no genera problemas. Eliminar un nodo negro requiere de reequilibrio.

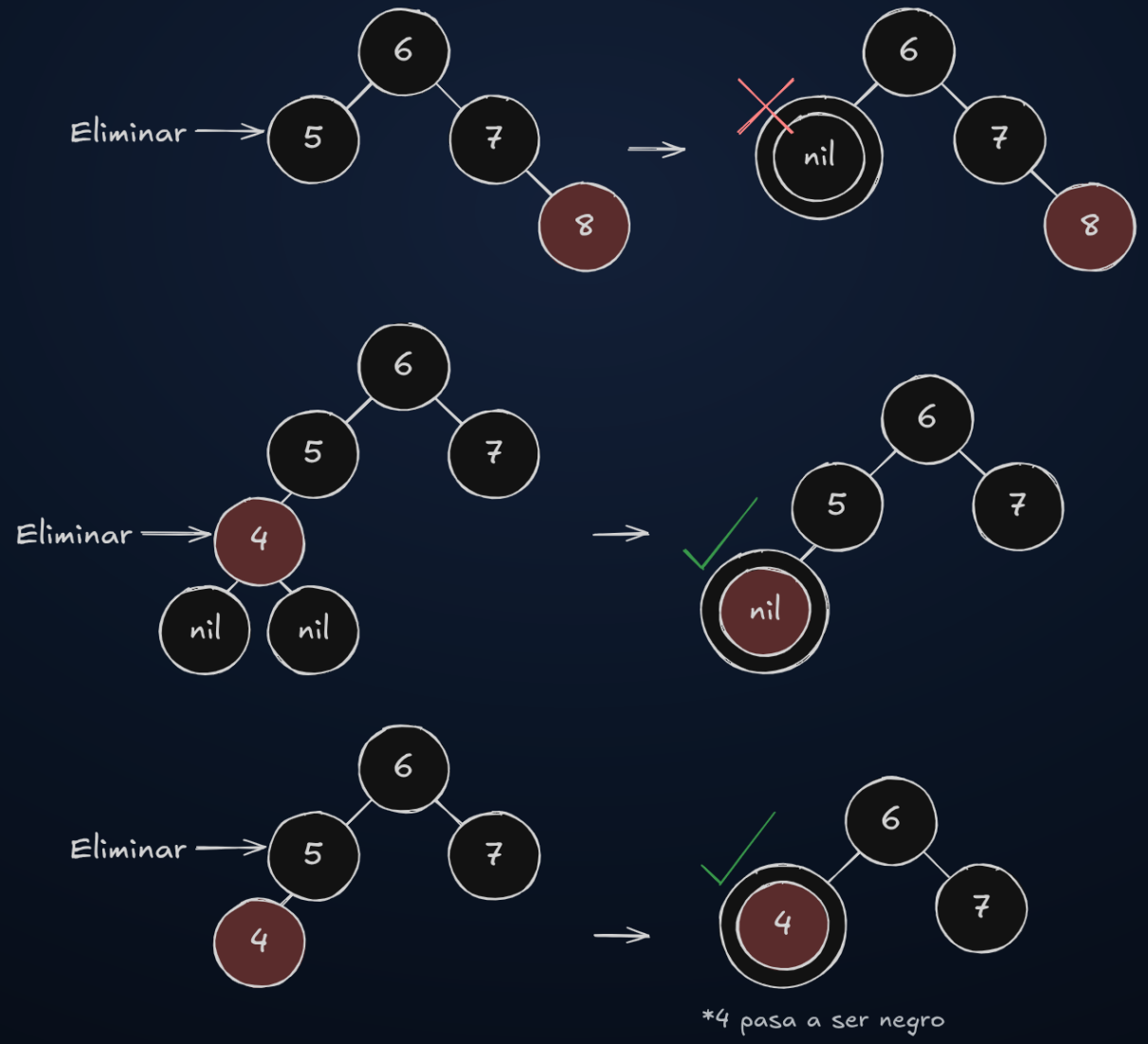
Concepto de "Doble Negro"

Al eliminar un nodo, en su lugar queda un único nodo nulo que **adquiere el color del nodo previamente eliminado**.

Recordando que los nodos nulos son **siempre negros**, si el nodo del que hereda su color es también negro, este se vuelve **doblemente negro**. El objetivo del borrado es eliminar ese "doble negro" y mantener una configuración válida del árbol mediante rotaciones y cambios de colores.

Notar que si el nodo eliminado es rojo, el nodo nulo **no se vuelve rojo**, sino que mantiene su color.

Borrado: Doble Negro



Borrado: `Doble Negro`

Llamaremos `u` al nodo doblemente negro, `s` su hermano y `r` al hijo rojo de `s` (si es que existe). Si `s` es **negro** se tienen los siguientes casos:

Left-Left

Si `s` es **hijo izquierdo** y `r` es **hijo izquierdo** o ambos hijos de `s` son rojos:

- Se realiza una rotación simple izquierda sobre `s`, `r` pasa a ser **negro**. En caso de que ambos hijos de `s` sean rojos, su hijo izquierdo pasa a ser **negro**.

Left-Right

Si `s` es **hijo izquierdo** y `r` es **hijo derecho**:

- Se realiza una rotación doble derecha-izquierda sobre `s`, `r` pasa a ser **negro**.

`Las rotaciones son equivalentes a las del AVL`

Borrado: `Doble Negro`

Llamaremos `u` al nodo doblemente negro, `s` su hermano y `r` al hijo rojo de `s` (si es que existe). Si `s` es **negro** se tienen los siguientes casos:

Right-Right

Si `s` es **hijo derecho** y `r` es **hijo derecho** o ambos hijos de `s` son rojos:

- Se realiza una rotación simple derecha sobre `s`, `r` pasa a ser **negro**. En caso de que ambos hijos de `s` sean rojos, su hijo derecho pasa a ser **negro**.

Right-Left

Si `s` es **hijo derecho** y `r` es **hijo izquierdo**:

- Se realiza una rotación doble izquierda-derecha sobre `s`, `r` pasa a ser **negro**.

`Como se puede ver, si s tiene ambos hijos rojos se puede hacer tanto el caso Left-Left como Right-Right`

Borrado: `Doble Negro`

Llamaremos `u` al nodo doblemente negro, `s` su hermano y `r` al hijo rojo de `s` (si es que existe). Si `s` es **negro** se tienen los siguientes casos:

Recoloring

Si `s` tiene **ambos hijos negros** se debe recolorear `s`, es decir, el padre de `s` **adquiere su color** y `s` pasa a ser **rojo**.

- Si el padre es **rojo**, entonces pasa a ser negro y el árbol queda equilibrado.
- Si el padre es **negro**, entonces pasa a ser **doble negro**, por lo que se debe repetir el proceso desde este punto.
  - Si el doble negro es la **raíz**, se transforma en un negro simple directamente.

Borrado: `Doble Negro`

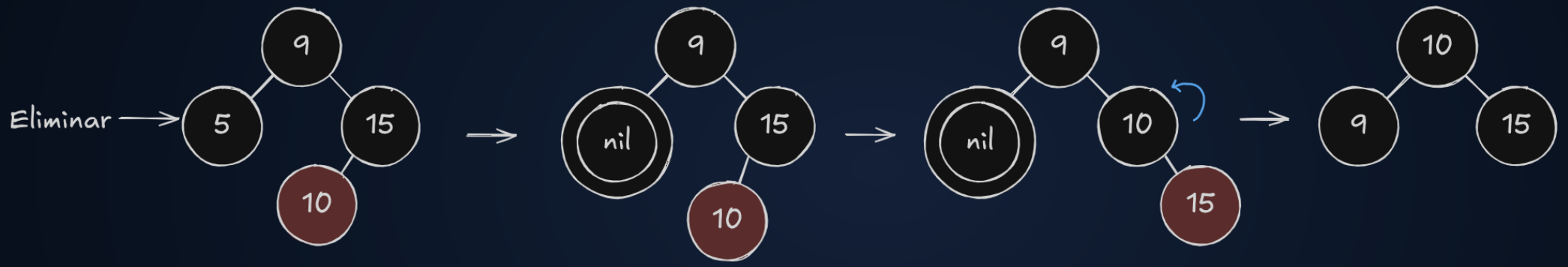
Llamaremos `u` al nodo doblemente negro, `s` su hermano.

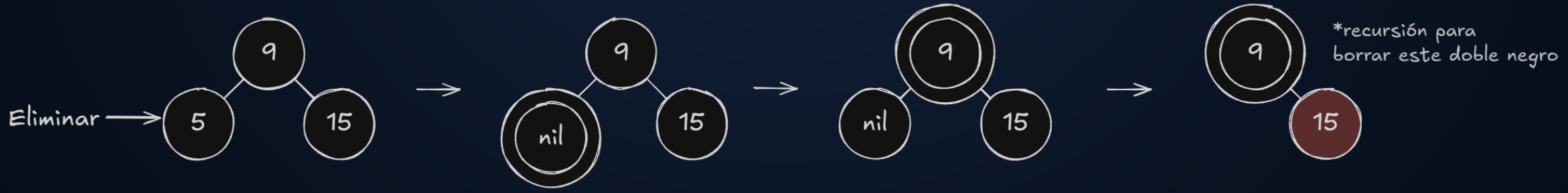
Si `s` es **rojo** entonces se debe realizar una **rotación simple** dependiendo de la posición de `s` (si `s` es hijo izquierdo, entonces se realiza rotación simple izquierda y viceversa).

Luego, se debe **recolorear** `s` y su padre de tal manera que `s` pasa a ser **negro** y su padre **rojo**. Esto causará que el nuevo hermano de `u` **siempre sea negro**.

Finalmente, se recolorean `u` y su nuevo `s` de tal manera que pasan a ser **rojos** y su `u` transfiere su color a su padre (el cual es rojo), por lo que este pasa a ser negro y el árbol queda equilibrado.

Ejemplos


Eliminar
9
5
15
10
nil
9
15
10
nil
9
10
15
10
9
15


Eliminar
9
5
15
nil
9
15
nil
9
15
9
15
*recursión para
borrar este doble negro


Eliminar
9
5
15
10
17
nil
9
15
10
17
15
9
17
nil
10
15
9
17
10

BLOQUE 2

Ejercicios

Ejercicio 1

Inserte la siguiente secuencia de números:

$$\{18, 31, 56, 82, 44, 73, 22, 75\}$$

En un AVL, Árbol 2-3 y un Árbol Rojo-Negro **inicialmente vacíos**, muestre claramente el paso a paso.

Ejercicio 2

Implemente la función `mostRecent`, que recibe una lista enlazada `fechas`, la cual contiene una colección desordenda de fechas. La función debe retornar un **arreglo con las fechas ordenadas en forma decreciente** (más reciente al más antiguo). La lista debe ser recorrida **una sola vez**.

Ejercicio 3

Implemente la función `kLowest`, que recibe la raíz de un árbol y un entero `k`. La función debe retornar el **k-ésimo menor número** almacenado en el árbol. Puede agregar los parámetros que estime necesarios a la función.

Por ejemplo, si el árbol almacena los números: $\{40, 20, 60, 10, 30, 50, 70\}$ y se llama a la función con `k = 3`, entonces la función debe retorrnar **30**.

CIERRE

¡Eso es todo! 